



تتمة في لغة SQL

د. سعيد دسوقي

قواعد معطيات

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 14/05/2026

ذكرنا في نهاية المحاضرة السابقة عبارات الجبر العلاقتي التالية كأمثلة سنقوم بتحويلها إلى تعليمات SQL:

1. $\sigma_{salary \geq 85000}(instructor)$

```
Select * from instructor where salary>=85000;
```

2. $\Pi_{ID,salary}(instructor)$

```
Select ID, salary from instructor;
```

3. *instructor* ⋈ *department*

Select * from instructor Natural Join department;

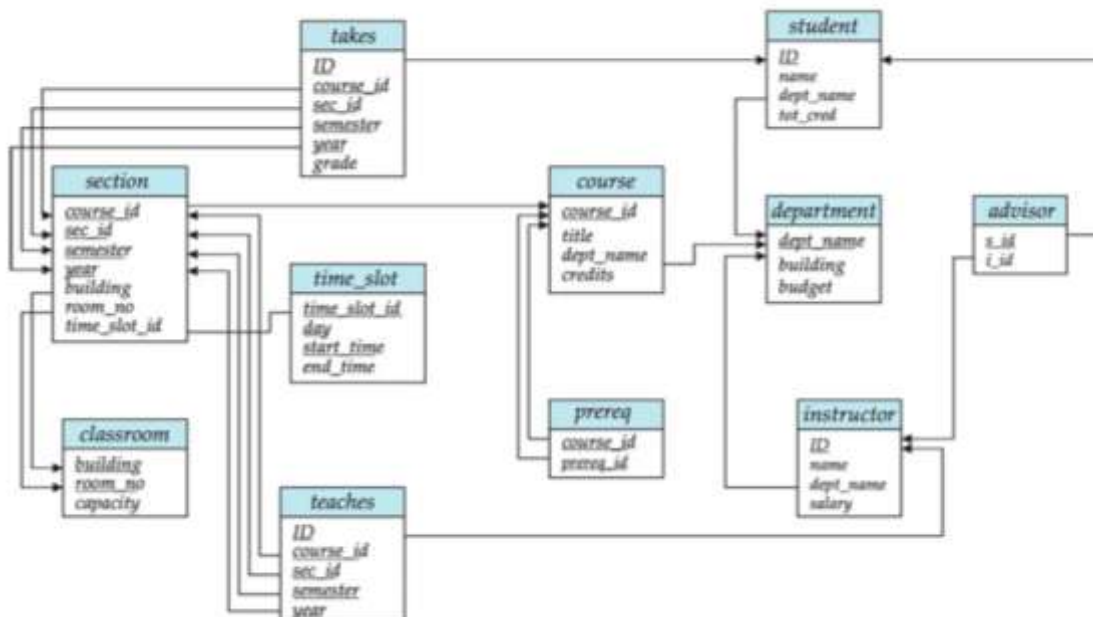
4. *instructor* $\times$ *department*

Select * from instructor Cross Join department;

5. $\Pi_{name}(instructor) \cup \Pi_{name}(student)$

Select name from instructor Union Select name from department;

مخطط بنية قاعدة معطيات في الجامعة:



المدرسون والأقسام:

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

الموارد ومعلوماتها:

course_id	title	dept_name	credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

العمليات على سلاسل المحارف

تستخدم هذه العمليات للتعامل مع النصوص وتعديل طريقة ظهورها أو البحث بداخلها بدقة أكبر، وينطبق ذلك على الأعمدة أو الثوابت.

1- مطابقة السلاسل المحرفية باستخدام LIKE (مطابقة تقليدية):

تستخدم للبحث عن "نمط" معين (Pattern) داخل النص وليس مطابقة كاملة، وتعتمد على رمزين أساسيين:

- الرمز (%): يطابق أي عددٍ من المحارف (صفر محرف أو أكثر) - بعض الأنظمة تستخدم الرمز " * " بدلاً عنه.
- الرمز (_): يطابق حرفاً واحداً فقط. - بعض الأنظمة تستخدم الرمز " ? " بدلاً عنه.

■ مثال:

```
select name
from instructor
where name like '%dar%'
```

هذا الاستعلام سيعيد أي اسم يحتوي على الأحرف "dar" في أي مكان (في البداية، المنتصف، أو النهاية).

■ مثال:

```
select name
from instructor
where name like '_s%'
```

هذا الاستعلام سيعيد أي اسم ثاني حرف فيه هو "s".

■ مثال:

```
select name
from instructor
where name like '____'
```

هذا الاستعلام سيعيد أي اسم مكون من 4 حروف.

ملاحظة هامة :

(إلغاء عمل الرموز الخاصة): إذا أردنا البحث عن رمز (%) أو (_)

كجزء من النص نفسه وليس كرمز مطابقة، نستخدم تعليمات escape.

مثال: للبحث عن نص "%100" نكتب:

```
like '100 \% ' escape '\'
```

ليتم اعتبار ال% حرفاً عادياً، فتعليلة escape دلت على أن المحرف \ للإلغاء.

2- دمج السلاسل (Concatenation):

تستخدم التعليمة (||) لدمج سلسلتين أو أكثر لتصبح عموداً واحداً، مثل دمج كلمة "السيد المحترم" مع اسم الموظف من الجدول لتظهر النتيجة "السيد المحترم [الاسم]" بشكل تلقائي لجميع الموظفين.

■ مثال:

```
select first_name || ' ' || father_name || ' ' || last_name as "Full Name"
from employee
```

سيظهر الاسم الثلاثي للموظفين بعمود واحد باسم "Full Name"، وقمنا بإضافة ' ' بينها كمسافات حتى لا يظهر الاسم الثلاثي متلاصقاً.

3- التحويل بين الأحرف الكبيرة والصغيرة:

upper: تحول النص للأحرف الكبيرة.

lower: تحول النص للأحرف الصغيرة.

لماذا نستخدمها؟

نستخدمها عندما لا نعرف كيف تم تخزين البيانات في الجدول (هل هي كبيرة أم صغيرة). فبدلاً من البحث عن "Biology" وقد تكون مخزنة "biology"، نقوم بتحويل الطرفين إلى حالة واحدة لضمان المطابقة.

■ مثال:

```
where upper(dept_name) = 'BIOLOGY'
```

ملاحظة:

هذه التوابع تغير شكل البيانات في "نتائج الاستعلام" فقط ولا تغير البيانات الأصلية المخزنة في الجداول.

4- طول السلسلة واقتطاع جزء منها:

تابع length: يعيد عدد المحارف في السلسلة.

■ مثال:

بدلاً من كتابة أربعة رموز _ _ _ _ للبحث عن اسم من 4 حروف، يمكننا كتابة:

```
where length(name) = 4
```

تابع substring: يستخدم لاقتطاع جزء من النص بدءاً من موقع معين وبطول محدد:

ملاحظة تقنية:

نقطة البداية (هل نبدأ العد من 0 أم 1) قد تختلف بين أنظمة قواعد البيانات مثل Oracle SQL Server.

■ مثال تطبيقي:

لو أردنا أخذ أول حرفين من كلمة "Computer" بدءاً من الحرف الثالث:
 "mp" substring('Computer', 3, 2) <- النتيجة ستكون

عمليات المجموعات (Set Operations)

تستخدم هذه العمليات لدمج نتائج استعلامين (أو أكثر) مع بعضهما، وهي تشبه تماماً العمليات التي درسناها في الجبر والمجموعات.

1. الاتحاد (union):

تستخدم لإيجاد العناصر الموجودة في الاستعلام الأول أو الاستعلام الثاني.

■ مثال: إيجاد المواد التي أعطيت في خريف 2009 أو ربيع 2010:

```
(select course_id from section where sem = 'Fall' and year = 2009)
```

union

```
(select course_id from section where sem = 'Spring' and year = 2010)
```

2. التقاطع (intersect):

تستخدم لإيجاد العناصر الموجودة في الاستعلام الأول والاستعلام الثاني في وقت واحد (المشتركة بينهما).

■ مثال: إيجاد المواد التي أعطيت في كلا الفصلين (خريف 2009 و ربيع 2010).

```
(select course_id from section where sem = 'Fall' and year = 2009)
```

intersect

```
(select course_id from section where sem = 'Spring' and year = 2010)
```

3. (except):

تستخدم لإيجاد العناصر الموجودة في الاستعلام الأول وليست موجودة في الاستعلام الثاني.

■ مثال: المواد التي أعطيت في خريف 2009 ولكنها لم تُعطَ في ربيع 2010.

```
(select course_id from section where sem = 'Fall' and year = 2009)
```

except

```
(select course_id from section where sem = 'Spring' and year = 2010)
```

تذكرة بالشروط الأساسية للعمليات على المجموعات:

- نفس عدد الأعمدة:
يجب أن ينتج عن كل استعلام نفس عدد الأعمدة.
- نفس نوع البيانات (Data Type):
يجب أن تكون أنواع الأعمدة متوافقة (مثلاً عمود نصي مع عمود نصي)، ولا يشترط أن يكون لها نفس الاسم.

ملاحظات هامة حول التكرار (Duplicate Handling):

يوجد فرق جوهري بين المجموعات وتعليمات SQL العادية:

1. الحذف الآلي للتكرار:
عمليات المجموعات (union, intersect, except) تقوم بحذف التكرار تلقائياً من النتائج، على عكس تعليمة select العادية التي تبقي التكرار إلا إذا أضفنا distinct.

2. إبقاء التكرار:

إذا أردت رؤية العناصر المكررة في النتيجة، يجب إضافة كلمة all بعد العملية:

```
union all
intersect all
except all
```

القيم المعدومة Null Values

تستخدم القيمة null للدلالة على أن القيمة غير موجودة، أو غير معروفة، أو غير واردة.
إن الـ null تختلف تماماً عن الصفر، فالصفر قيمة معروفة أما الـ null فهي غياب للقيمة.

1. كيفية فحص القيم المعدومة:

لا يمكننا استخدام المساواة العادية (=) مع الـ null لأنها ليست قيمة طبيعية، بل نستخدم الشرط is null.

■ مثال:

لإيجاد أسماء المدرسين الذين رواتبهم غير مدخلة في النظام:

```
select name
from instructor
where salary is null;
```

2. العمليات الحسابية مع ال Null:

أي عملية رياضية (جمع، طرح، ضرب، قسمة) يكون أحد أطرافها null فإن النتيجة تكون دائماً null.
مثال:

$\text{null} = \text{null} + 5$

3. منطق القيم الثلاث (Three-valued logic):

بما أن هناك قيمة "غير معروفة" (unknown)، فإن العمليات المنطقية في SQL تعتمد على ثلاثة احتمالات (صح، خطأ، غير معروف):

■ في حالة OR (أو):

(unknown or true) تعطي true (لأن وجود طرف واحد صحيح يكفي).

unknown or false تعطي (unknown).

unknown or unknown تعطي (unknown).

■ في حالة AND (و):

true and unknown تعطي (unknown).

false and unknown تعطي false (لأن وجود طرف واحد خاطئ يجعل النتيجة خاطئة فوراً).

unknown and unknown تعطي (unknown).

■ في حالة NOT (النفى):

النفى ال unknown يبقى unknown.

4. المقارنات ونتائج الاستعلام:

نتائج المقارنة: إذا قارنا أي قيمة مع null (مثل: $\text{salary} > 50000$) وكان الراتب null فإن نتيجة المقارنة تكون unknown.

5. قاعدة ال Where:

تعليمية where تعتبر النتيجة "غير المعروفة" (unknown) بمثابة false.
أي أن السطر الذي ينتج عن شرطه قيمة unknown لن يظهر في نتائج البحث.

■ مثال توضيحي:

لو كان لدينا مدرس راتبه null وكتبنا الشرط `where salary > 10000` النتيجة ستكون unknown وبالتالي لن يظهر هذا المدرس في النتائج، مع أننا لا نعرف فعلياً هل راتبه أكبر أم أصغر.

توابع التجميع (Aggregate Functions)

سميت بهذا الاسم لأنها تأخذ مجموعة من القيم (مجموعة أسطر) كمداخلات، وتقوم بمعالجتها لتعيد قيمة واحدة فقط كـ "تجميع" لها.

التوابع الأساسية:

avg: لحساب المتوسط الحسابي للقيم.

min: لإيجاد أصغر قيمة.

max: لإيجاد أكبر قيمة.

sum: لحساب مجموع القيم.

count: لحساب عدد القيم (الأسطر).

أمثلة الاستخدام:

1. حساب متوسط رواتب مدرسي قسم معين:

```
select avg(salary)
from instructor
where dept_name = 'Comp. Sci';
```

2. حساب عدد مدرسي قسم معين (أو عدد الصفوف الكلي):

نستخدم `count(*)` لعدّ كافة الأسطر المحققة للشرط، مثلاً عد جميع المواد:

```
Select count (*)
from course;
```

3. استخدام distinct مع التجميع:

نستخدمها لعدّ العناصر الفريدة بدون تكرار.

مثال:

لو أردنا معرفة "عدد المدرسين" الذين أعطوا مواداً في فصل معين، نستخدم `count(distinct ID)`؛ لأنه لو أعطى المدرس أكثر من مادة سيحسب "واحد" فقط، وهذا هو المنطقي.

```
select count (distinct ID)
from teaches
where semester = 'Spring' and year = 2010;
```

ملاحظات هامة جداً:

أولاً: التعامل مع القيم المعدومة (null):

توابع التجميع تتجاهل قيم الـ null تماماً ولا تدخلها في الحساب.

في المجموع (sum):

إذا كان هناك 10 مدرسين وأحدهم راتبه null: سيجمع الرواتب التسعة الباقية فقط (ولن تكون النتيجة null كما في العمليات الحسابية العادية).

في المتوسط (avg):

سيجمع الرواتب الـ 9 الموجودة، ويقسم المجموع على 9 فقط وليس على 10.

في العدد (count):

إذا كتبت count(salary) فإنه لن يُعدّ الشخص الذي راتبه null أما إذا كتبت count (*) فإنه سيعدّ جميع الأسطر بغض النظر عن وجود قيم null.

ثانياً: شكل النتيجة: الاستعلام الذي يحتوي على تابع تجميع (Group by)

يعيد دائماً سطرًا واحدًا فقط وقيمة واحدة (أو عدة قيم تجميعية بجانب بعضها).

ثالثاً: دمج أكثر من تابع: يمكننا طلب أكثر من معلومة تجميعية في نفس الوقت:

وهنا ستعرف كم موظف "لديه راتب" (الأولى) وكم موظف "موجود بالشركة" (الثانية).

```
select count(salary), count(*) from instructor;
```



التجميع بحسب الوصفة (Group By)

تستخدم تعليمة group by عندما نريد تطبيق توابع التجميع (مثل المتوسط أو المجموع) على "مجموعات فرعية" من البيانات وليس على الجدول ككل.

■ المفهوم الأساسي:

بدلاً من الحصول على "رقم واحد" يمثل متوسط رواتب كل المدرسين، تسمح لنا group by بالحصول على متوسط رواتب مدرسي "كل قسم على حدة".

مثال :

ID	name	dept_name	salary
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

dept_name	salary
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

```
select dept_name, avg(salary)
from instructor
group by dept_name;
```

في هذا المثال، سيقوم النظام بتجميع المدرسين الذين ينتمون لنفس القسم في مجموعة واحدة، ثم يحسب المتوسط لكل مجموعة.

■ القاعدة الذهبية لل Group By (مهمة للامتحان):

قاعدة تقنية في SQL: "أي عمود يظهر في تعليمات select ولا يُستخدم داخل تابع تجميع، يجب بالضرورة أن يظهر في قائمة group by".

لماذا؟ لا يوجد "تناسب"؛ فتابع التجميع يعيد "قيمة واحدة" للمجموعة، بينما الأعمدة العادية (مثل اسم المدرس) لها قيم متعددة، لذا لا يمكن للنظام عرض اسم مدرس واحد بجانب متوسط رواتب القسم بأكمله إلا إذا تم التجميع حسب ذلك الاسم.

نتيجة الخطأ:

إذا كتبت عموداً في select ونسيته في group by سيعطيك النظام خطأ في القواعد (syntax error).

ملاحظات تقنية:

التجميع حسب أكثر من واصفة:

يمكننا التجميع حسب أكثر من عمود (مثلاً حسب القسم والجنس معاً).

التجميع حسب المفتاح الرئيسي (ID):

إذا قمنا بالتجميع حسب ال-ID (وهو قيمة فريدة لكل مدرس)، فإن تابع التجميع (مثل avg) سيعطيك نفس قيمة الراتب لهذا المدرس، لأن كل مجموعة ستحتوي على شخص واحد فقط.

شكل النتيجة:

نتيجة الاستعلام هنا تكون جدولاً كاملاً يحتوي على عدة أسطر (سطر لكل مجموعة)، على عكس التجميع البسيط الذي يعيد سطرًا واحد.

ترتيب التنفيذ:

النظام يقوم أولاً بتنفيذ from ثم where (لاختيار الأسطر)، وبعدها يأتي دور group by لتقسيم تلك الأسطر المحققة للشرط إلى مجموعات، وأخيراً يتم حساب التابع التجميعي.



الشروط على التجميع (Having Clause)

تستخدم تعليمة having لنميز بين "شروط على الأسطر" (Where) و"شروط على المجموعات" (Having). فنحن نستخدمها عندما نريد وضع شرط يعتمد على تابع تجميعي.

الفرق بين having و where (سؤال امتحاني):

where: يطبق على الأسطر المنفردة قبل عملية التجميع.
having: يطبق على المجموعة ككل بعد حساب تابع التجميع.

مثال:

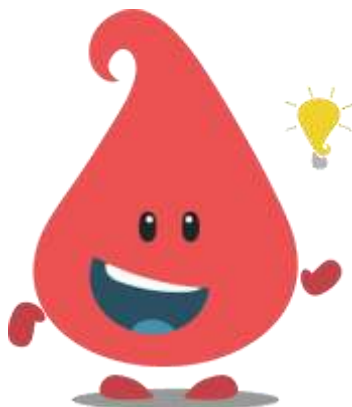
ID	name	dept_name	salary
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

dept_name	salary
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

لو أردنا فقط الأقسام التي "متوسط" رواتب مدرسيها أكبر من 42,000:

```
select dept_name, avg(salary)
from instructor
group by dept_name
having avg(salary) > 42000;
```

هنا، الأقسام التي متوسط رواتبها أقل من هذا الرقم (مثل Music و History) سيتم حذفها من النتائج النهائية.



ترتيب تنفيذ تعليمات SQL (مهم جداً):

طريقة ترتيب "محرك قاعدة البيانات" من الداخل عند كتابة استعلام معقد، والترتيب هو:

1. from: يحدد الجداول ويتأكد من الصلاحيات.
2. where: يختار الأسطر التي تحقق الشرط.
3. group by: يقسم الأسطر المتبقية إلى مجموعات.
4. حساب التوابع التجميعية: يحسب (Avg, Sum, Max ...) لكل مجموعة (بالتوازي مع الخطوة السابقة).
5. having: يطبق الشرط على المجموعات الناتجة (لذلك لا يمكن وضع شرط تجميعي في where لأن الحساب لم يكن قد بدأ بعد).
6. select: يختار الأعمدة المطلوب عرضها.
7. order by: يرتب النتائج النهائية.

(Nested Sub-queries) الاستعلامات الجزئية المتداخلة:

ببساطة هي عبارة عن "سؤال داخل سؤال"، أو تعبير select-from-where موجود داخل where لاستعلام خارجي. نستخدمها غالباً للقيام بعمليات معقدة مثل فحص الانتماء لمجموعة أو مقارنة القيم مع مجموعة كاملة.

■ اختبار الانتماء للمجموعات باستخدام in و not in:

تستخدم in للتأكد من أن القيمة التي نبحث عنها "موجودة" ضمن نتائج الاستعلام الداخلي.

■ مثال:

(بديل للتقاطع): إيجاد المواد التي أعطيت في خريف 2009 وربيع 2010.

[الاستعلام الداخلي:](#)

يخرج لنا قائمة بـ 2010 مواد ربيع 2010.

```
select distinct course_id
from section
where semester = 'Fall' and year= 2009 and
       course_id in (select course_id
                     from section
                     where semester = 'Spring' and year= 2010);
```

[الاستعلام الخارجي:](#)

يُمر على مواد خريف 2009، ويسأل: "هل هذا الرقم موجود في القائمة (الداخلية)؟".

ملاحظة من الدكتور: استخدام in غالباً ما يكون أسرع من عملية التقاطع (intersect): لأن النظام ينفذ الاستعلام الداخلي مرة واحدة ويضع نتائجه في الذاكرة ثم يقارن بها مباشرة.

■ مثال:

إيجاد عدد الطلاب الذين قام بإعطائهم الأستاذ ذي الرقم 10101 بدون تكرار (يبحث ضمن المواد التي قام الأستاذ بتدريسها عن الطلاب الذين سجلوا بها):

```
select count (distinct ID)
from takes
where (course_id, sec_id, semester, year) in
      (select course_id, sec_id, semester, year
       from teaches
       where teaches.ID= 10101);
```

استخدام not in:

تستخدم لإيجاد العناصر غير الموجودة، وهي تعمل تماماً مثل عملية الفرق (except).

■ مثال:

```
select distinct course_id
from section
where semester = 'Fall' and year= 2009 and
      course_id not in (select course_id
                       from section
                       where semester = 'Spring' and year= 2010);
```

■ مقارنة المجموعات باستخدام some و all:

أحياناً نريد مقارنة قيمة واحدة مع "مجموعة قيم" (وليس قيمة واحدة فقط)، وهنا نستخدم هذه الأدوات:

الأداة some (أو any):

تعني أن الشرط يكفي أن يتحقق مع عنصر واحد على الأقل من المجموعة.

■ مثال:

أوجد أسماء المدرسين الذين يقبضون راتباً أعلى من أحد مدرسي قسم النظم المعلوماتية:

```
select name
from instructor
where salary > some (select salary
                    from instructor
                    where dept name ='Comp.Sci.')
```

الأداة all:

تعني أن الشرط يجب أن يتحقق مع جميع عناصر المجموعة بلا استثناء.

■ مثال:

أوجد أسماء المدرسين الذين يقبضون راتباً أعلى من جميع مدرسي قسم النظم المعلوماتية

```
select name
from instructor
where salary > all (select salary
                    from instructor
                    where dept name = 'Biology');
```

ملاحظة ربط الاستعلامات المتداخلة بتوابع التجميع:

أوضح الدكتور أننا نستطيع كتابة نفس الاستعلامات بطريقتين، وكلاهما صحيح برمجياً:

1. باستخدام المجموعات: (select ... salary > some)

2. باستخدام التجميع: (select min(salary) from ... salary >)

لماذا؟ لأنك إذا كنت أكبر من "أحد" الرواتب، فأنت بالضرورة أكبر من "أقل" واحد فيهم. وإذا كنت أكبر من "كل" الرواتب، فأنت بالضرورة أكبر من "أعلى" واحد فيهم.

ملاحظات تقنية هامة:

توافق الأعمدة:

عند المقارنة، يجب أن يكون عدد الأعمدة ونوع البيانات في الاستعلام الخارجي مطابقاً تماماً لما ينتجه الاستعلام الداخلي.

التعامل مع النصوص:

يمكن استخدام هذه المقارنات مع النصوص أيضاً، حيث تتم المقارنة "أبجدياً" (مثلاً الحرف B أكبر من A).

النتائج المنطقية:

إذا كان الاستعلام الداخلي فارغاً (لا توجد نتائج)، فإن all تعطي دائماً true (لأنه لا يوجد أحد يخالف الشرط)، بينما some تعطي false.

توضيح لكيفية عمل الـ All والـ Some:

الأداة some (تعني: واحد على الأقل)

يكفي أن يتحقق الشرط مع عنصر واحد فقط من المجموعة لتكون النتيجة النهائية true.

■ أمثلة:

True $\leftarrow$ (some {0, 5, 6} > 5) (لأن 5 أصغر من 6).

False $\leftarrow$ (some {0, 5} > 5) (لأن 5 ليست أصغر من أي عنصر بالمجموعة).

True $\leftarrow$ (some {0, 5} = 5) (لأن 5 تساوي أحد العناصر).

True $\leftarrow$ (some {0, 5} $\neq$ 5) (لأن 5 لا تساوي أحد العناصر "0").

(5 < some

0
5
6

) = true

(5 < some

0
5

) = false

(5 = some

0
5

) = true

(5 $\neq$ some

0
5

) = true (since 0 $\neq$ 5)

ملاحظة:

العملية (some =) هي تماماً نفس العملية in.

لكن انتبه: العملية (some $\neq$) لا تساوي not in.

2- الأداة all (تعني: الجميع بلا استثناء):

يجب أن يتحقق الشرط مع كافة عناصر المجموعة لتكون النتيجة النهائية true.

(5 < all

0
5
6

) = false

(5 < all

6
10

) = true

(5 = all

4
5

) = false

(5 $\neq$ all

4
6

) = true (since 5 $\neq$ 4 and 5 $\neq$ 6)

False $\leftarrow$ (all {0, 5, 6} > 5) (لأن 5 ليست أصغر من الصفر ولا من نفسها).

True $\leftarrow$ (all {6, 10} > 5) (لأن 5 أصغر من الـ 6 وأصغر من الـ 10 أيضاً).

True $\leftarrow$ (all {4, 6} $\neq$ 5) (لأن 5 لا تساوي الـ 4 ولا تساوي الـ 6).

ملاحظة:

العملية (all $\neq$) هي تماماً نفس العملية not in.

لكن انتبه: العملية (all =) لا تساوي in.

بعض الملاحظات المهمة:

■ الربط مع توابع التجميع:

تستطيع فهم هذه العمليات (all, some) بربطها بالـ min والـ max:

- أكبر من some: تعني أنك أكبر من أصغر قيمة (Min) في المجموعة.
- أكبر من all: تعني أنك أكبر من أكبر قيمة (Max) في المجموعة.

- المقارنة الأبجدية: هذه المقارنات تعمل أيضاً مع النصوص (Strings) بناء على الترتيب الأبجدي
- السرعة والأداء: استخدام الاستعلامات المتداخلة (in, some, all) غالباً ما يكون أسرع من دمج الجداول الكبيرة أو استخدام عمليات المجموعات التقليدية لأن النظام يعالج المجموعة الداخلية مرة واحدة.

- انتهت المحاضرة -



**سوف نبني حلماً كان كنزاً فينا
سنعيشه دوماً فالأمل يقويننا!**